# PowerCloud Roll Out

## How the practice works now: the Power Cloud OS

All-hands, Thursday, September 3, 2026

Internal. Client names are fine in this room.

Power Cloud
CONSULTING

# Why this meeting is mandatory

- Today the OS stops being a pilot. From here on it is how Power Cloud does client work: assessments, implementations, builds, support tickets, status reporting, data work.
- You leave knowing what it does end to end, having watched three colleagues show real work with it, and knowing exactly what you are expected to do next week.
- What stays with you: judgment, proofreading, the numbers, the client relationship. What moves to the tool: the typing and the memory.

**90 minutes: the system, one engagement end to end, three case studies, seven announcements, one ask.**

# Run of show

| Time | Segment | Who |
|---|---|---|
| **0:00** | Open. What changes today. | Mike |
| **0:05** | What the OS is, how you get in, and your first session in Claude Code. Live on screen. | Mike |
| **0:17** | What it has already done, then one engagement end to end with a real folder on screen. | Mike |
| **0:40** | Three case studies, ten minutes each. Real PS work, one client, one problem. | Imran, Justin, Ron |
| **1:10** | Seven announcements. | Mike |
| **1:22** | The ask, then open floor. | Mike |

01

# What the Power Cloud OS is, and how you get in

For anyone opening Claude Code for the first time. The how-to PDF covered setup; this is the part after setup.

Power Cloud
CONSULTING

# The OS is three things. You touch two of them.

## A shared drive of client folders

- G:\Shared drives\Power Cloud OS. One folder per client, the same shape every time.
- Everything about a client lives in their folder. That folder is the memory.
- You work here every day.

## Claude Code, opened inside a folder

- The Code tab of the Claude desktop app. A chat attached to a folder: it reads and writes the files there, runs tools, and connects to NetSuite, Asana, Gmail and Drive.
- You talk to it in plain English. It does the typing and the digging.
- You work here every day.

## The written-down way we work

- Playbooks, writing standards, checklists, skills: markdown files at the root of the drive.
- Claude reads them automatically at the start of every session and stays inside their rules on its own.
- You never need to open them.

**Plainly: open Claude Code in a client's folder and say what you need. Everything else is machinery to make that safe and consistent.**

# If this is your first time in Claude Code

- It is a chat window attached to a folder. What makes it different from Claude in a browser: it can read and write the files in that folder, run scripts and connectors, and it loads the workspace rules automatically.
- You type plain English. There are no commands to learn. The prompts on these slides are starting points; say it your way.
- It works in the open. You watch it read files, run queries and write drafts, one step at a time in the conversation. You can interrupt and redirect at any point.
- It asks before anything with consequences. A permission prompt is a question, not an error. Allow file edits in the client's working/ folder. Decline a production write during discovery, every time.
- A session is one conversation. Start a new one for each task. Nothing is lost when you do: everything it wrote is in the folder, and the next session reads it.

**You are managing, not typing. Give it the job, check its work, own what goes out.**

# How you get in

1

### Open Claude

The Claude desktop app, then the Code tab at the top of the window.

2

### Open the client's folder

G:\Shared drives\Power Cloud OS\{Client}. Always a client folder. Never the root, never a personal folder.

3

### Run session start

"This is the {Client} engagement. Run session start." It reads the fact sheet, checks the NetSuite account, lists its tools.

4

### Name the session

Paste the /rename line it suggests: client plus environment.

5

### Say what you need

A question, a draft, the exec summary, a ticket, the assessment. The menu is in the appendix.

Check you are signed into your Power Cloud account, not a personal one. The wrong account loses the team's context and connections.

# What you will see on screen

## While it works

- Your message, then Claude's steps as it works: the files it reads, the queries it runs, the drafts it writes. Collapsed by default; expand any step to see exactly what it did.
- Permission prompts when it wants to do something with consequences. Read them before you answer.
- Its answer, and the files it produced, which appear in the client folder in Explorer as it goes.

## What you do

- Let it work. Waiting is not billable, so line up parallel work or open a second session for another client.
- Answer when it asks. It interviews you when it hits a gap instead of guessing.
- Correct it out loud when it gets something wrong, and tell it to remember.
- Read everything before you trust it. Polished output at speed is exactly what makes skipping review tempting.

**Auto-accepting file edits in a working session is fine. Bypassing permissions never is.**

# Your first ten minutes, zero risk

- Pick a client you know well and open their folder.
- Run session start. Watch what it reads, and what it confirms about the connected account.
- Ask a question only someone who had read the whole folder could answer.
- Then challenge one claim with "what's your source?" and make it trace the answer back to a file or a query.
- Everything in this exercise is read-only. Nothing reaches the client and nothing touches NetSuite.

**WHAT YOU SAY**

```
> This is the Teppermans engagement. Run session start.

> Based on everything in this folder, what should I know before my next call with them?

> What's your source for that?
```

# The one idea: the folder is the memory

```
{ClientName}/
├── CONTEXT.md   living fact sheet
├── .claude/     settings, render gate
├── source/      client material, read-only
├── working/     evidence, drafts, LOE log
├── deliverables/  finals only
└── execution/   build work, sandbox first
```

- Everything about a client lives in that client's folder: source material, working evidence, finals, build work.
- Nothing important lives in a chat history, in a consultant's head, or in the model's context.
- Any consultant who opens the folder and starts a session is talking to something that already understands the engagement.
- Clear a session, swap a machine, let a week pass. The state survives because it is on disk.
- Every client folder has the identical shape. The separation does real work: it keeps the client's words untouched, keeps half-finished thinking out of finals, and walls write-enabled build off from read-only discovery.

# What we own, and what we rent

## Layer one: the methodology. Ours.

- The folder structure, the skills, the reference files, the checkpoints, the evidence tiers.
- Written down, versioned, improved after every engagement.
- The reason the output comes up the same way regardless of who runs it.
- This layer is the asset.

## Layer two: the engine. Rented.

- Claude Code, plus the MCP connections into NetSuite, Celigo, Asana, Gmail and Drive.
- Swappable and model-portable on purpose.
- If a better engine appears, the methodology moves onto it without being rewritten.
- Nothing client-specific is hard-coded into a shared skill. It lives in that client's CONTEXT.md.

The competitive position is not "we use AI." Anyone can rent the engine. The position is a discovery process a machine executes the same way every time, with evidence discipline most firms do not apply even by hand.

# What it has already done

**10 to 4**

hours to generate a client report.

**8 to 2**

hours to produce a rapid assessment: from a transcript instead of from a consultant's notes.

**Under 1%**

Shinesty's actual return rate, against the 25 to 30% the client reported. Script-level proof nobody would have dug up by hand.

**1 day**

Teppermans: a suitelet fix went from call transcript to sandbox-deployed. Nobody touched the script but Claude.

The standout after several weeks of use has been data handling: messy conversation plus raw exports, landed in NetSuite in the right shape, better than expected. And the three case studies coming up are all work that happened inside the OS.

# Not a pilot: what is on the shelf today

**10**

clients with engagement records in the OS

**18**

engagement records, including nested custom projects

**31**

finals shipped from deliverables/ folders

**8**

engagements with execution work in progress

From ENGAGEMENT_LOG.md at the root, generated September 2 from what each folder actually wrote to disk. Nobody maintains it by hand, so it reflects work done rather than a field someone remembered to update.

02

# One engagement, end to end

Sales handover to retro. Walked from the how-to guide's own arc, with a real folder on screen.

Power Cloud
CONSULTING

# The arc

1

### Sales handover

The email that starts it. Folder created, sales notes dropped in.

2

### Access & consent pack

Disclosure, both environments, Script Reader, MCP request. One email, day one.

3

### Connected, identity-checked

Connection added centrally. Session confirms the account belongs to this client.

4

### Kickoff prep

Read-only discovery and website research before the client call.

5

### Discovery lands

Transcripts and exports into source/. Read-only, never edited.

6

### The assessment

Six stages. Candidate register, evidence tiers, three deliverables.

7

### Four checkpoints

Agenda, findings, drafts, LOE line by line. Client signs off.

8

### Execution

Sandbox first. Plan approved before build, one approval per production deploy.

9

### Exec summary

Before every status call. Human-reviewed, always.

10

### Retro

Learnings triaged into the shared library. The next engagement starts better.

# It starts with the sales handover

- Create a plain-name folder under G:\Shared drives\Power Cloud OS. No "RA" suffix; the folder outlives any one engagement.
- Drop whatever you have in it, loose is fine: sales notes, transcripts, exports, the SOW.
- Open Claude Code in that folder and say the line.
- Claude files the documents into source/, checks for an existing Asana project and folder-name variants before creating anything, interviews you (engagement type, contacts, stack, billing channel), pre-fills CONTEXT.md, links the legacy folder as read-only history, and drafts the access & consent pack for you to send.
- Old client, new work: same line, and it asks about their historical folder instead of copying it over.

**WHAT YOU SAY**

```
> New client. Run session start.

> This is new work for Lulus, an old
client. Set them up.

> Foundrae is an in-flight project we're
bringing into the workspace. Set them
up.
```

# The access & consent pack goes out on day one

- **Disclosure.** We use AI and connect to their NetSuite through MCP, read-only during discovery. A conversation and a written yes, logged in CONTEXT.md with the date.
- **Both environments, up front.** Production for discovery: current data, and the read-only rule means no write risk. Sandbox for the build later; it has lead time.
- **Script Reader install**, with the client's go-ahead (Mike, Taha or Dev). Without it, any claim about what a script does stays Reported, never Verified.
- **MCP connection added centrally.** Send Mike or Taha the account ID. Connections carry the environment in their name ("- Prod", "- SB"), and the write guardrail reads that name.
- **Shopify or QuickBooks in the stack?** The pack has blocks for those too. Credentials never land in this Drive.

**Before real work: disclosure approved, access granted, Script Reader installed, connection live, identity check passed.**

# Every session starts the same way

- Open the client's folder and say the line. Same starting point for every phase, discovery through execution.
- It reads CONTEXT.md, inventories the folder, confirms the connected NetSuite account actually belongs to this client, and checks which tools are present.
- If the account does not match, it stops and asks. Do not override; fix the connection.
- It ends by suggesting a session name. Paste it. Generic titles have already caused production and sandbox confusion.
- Then say what you need: prep, the assessment, execution, a ticket, the exec summary, a question with the client's full context behind it.

**WHAT YOU SAY**

```
> This is the Teppermans engagement. Run
session start.

> /rename Teppermans SB - suitelet fix

> Pick up where you left off.
```

# Kickoff prep happens before the client call

- Mines the sales notes and any transcripts, runs read-only MCP discovery against the account, and researches the client's public website.
- Produces a question list grounded in what the account shows, not a generic checklist, plus a time-boxed agenda.
- Gaps between what was said on the sales call and what is actually configured go to the top of the agenda.
- Checkpoint 1: the client-facing one-pager waits for your review before it goes anywhere.
- Taha's line from the summit: "Before the kickoff, they're asking you questions which nobody even asked."

**WHAT YOU SAY**

```
> We got the handoff from sales. Prep me
for the kickoff call, and draft the
questions we should send the client in
advance.

> Prep for the onsite.
```

# Discovery lands in source/, and source/ is sacred

- Transcripts, recordings, exports, client documents go into source/. That folder is the client's words and data.
- Never edit it. Transform via copies in working/. Never Explorer-copy a .gdoc; export it properly.
- Read-only during discovery, no exceptions, even on a live production account. Queries, record reads and metadata only.
- If a production-write confirmation ever appears during discovery work, the answer is decline. The guardrail is protecting a client account, not throwing an error.
- Two offerings, same machinery: **Rapid Assessment** onsite over two full days, **Fast Track Discovery** remote in two-hour chunks.

**Read-only during discovery. No exceptions, even on a live production account.**

# The assessment: six stages, each with a human check

| Stage | What it does | Produces | Human |
|---|---|---|---|
| **01 Pre-onsite prep** | Full read-only pass over the account so the questions are grounded in what it shows. | working/onsite_prep.md, one-page client agenda | Checkpoint 1 |
| **02 Evidence mining** | Every material claim logged with file, timestamp, speaker. Then an exhaustive sweep for candidate work items at a low threshold. | working/evidence.md, working/candidates.md | |
| **03 Verification** | Read-only queries confirm or refute each claim. Findings map to Power Cloud products, always with a native or third-party alternative named. | working/verification.md | Checkpoint 2, findings review |
| **04 Drafting** | Approved findings become three markdown drafts. Nothing renders yet. | Three drafts in working/ | |
| **05 Humanize, review, render** | Copy scan first, so you review finished prose. Draft review, then the LOE review line by line. Only then the branded finals. | working/loe_log.md, finals in deliverables/ | Checkpoints 3 and 4 |
| **06 Retro** | What the engagement taught the practice, walked with a human. Promote, keep local, or discard. | working/retro.md, updates to the shared library | Decision on every entry |

# Every claim is tiered

## Verified

Confirmed by a read-only query against the account. The query and its result are logged in working/verification.md.

## Reported

Stated on a call or in a client document. Not confirmed. Never presented as fact, no matter how confidently it was said.

## Inferred

Our analysis, or an industry pattern applied. Reasoning, not evidence. A health verdict is Inferred and says so.

The disagreement rule: where a call claim and the account disagree, the gap is itself a finding and gets called out rather than quietly reconciled. People misremember their own configurations. Tiers stay internal; they never clutter the client's document.

# Nothing falls off the list silently

- Surfacing every potential work item is the top priority of the whole process. It never depends on you remembering an item, or on a transcript being complete.
- Candidates are captured at a deliberately low threshold from every source into working/candidates.md: every pain point, passing complaint, manual workaround, and every anomaly found in the account itself.
- A candidate leaves the register in exactly three ways: it becomes a work item, it moves to fast-follow, or it is dropped with a written reason. Never by silent omission.
- Materiality is your call at the findings review, applied to a complete list, instead of a silent judgment made during extraction.
- Behind it, a per-process-area completeness checklist forces an explicit "not discussed" or "not applicable" for anything the sources never covered.

A candidate may never simply be absent. If it was said on a call or seen in the account, it stays on the list until a human decides otherwise.

# What always ships on a Recovery or an Implementation

| Deliverable | Job | Shape |
|---|---|---|
| **{Client}_Assessment_{date}.docx** | The source of truth. Findings, evidence, the complete work-item menu with effort and priorities, phasing, and the commercial framing. On an Implementation this is the roadmap: same evidence work, framed as milestones toward a target state. | .docx and PDF |
| **{Client}_Presentation_{date}.pptx** | The meeting vehicle. A twenty-page PDF cannot be reviewed live. Leads with the two or three highest-impact issues; talking points live in speaker notes. | .pptx and PDF |
| **{Client}_Execution_Plan.md** | The technical artifact a future session builds from. Per task: problem, affected objects, build classification, steps, acceptance criteria, sandbox test plan, rollback. | .md, internal, updated in place |

**Any net-new custom development, in any engagement type, also carries a Use Case the client signs before build and an FDD & TDD handover when it ships. A Custom Project ships those two instead of the three above.**

# Checkpoint 4: the numbers are yours

- Claude drafts every LOE figure. You own it, line by line, before anything renders. The hours review is called out explicitly, never assumed inside the draft review.
- Every line is logged in working/loe_log.md: the drafted figure, the approved figure, and a one-line reason for any change. That log is the calibration feed for LOE_BASELINES.md.
- Figures marked [no prior basis] get first attention.
- Commercial posture is a separate field from engagement type. **Pre-agreement**: LOE, phasing with hours, a recommended first phase, billing terms. **Under agreement**: none of that reaches the client; effort tracks internally only.

**LOE numbers are commercially binding once sent. Claude drafts them, a human owns them.**

# Client signs off. Execution builds sandbox first.

- Execution runs from {Client}/execution/, write-enabled, under its own guardrails. Discovery history sits one level up and stays reachable.
- **Gate 1, the plan.** A written plan per task, approved before any build. Three honest exits: build, a targeted question list for the client, or a throwaway sandbox spike. Guessing a design decision is not one of them.
- **Routes**, in preference order: SDF object (versioned, diffable, revertible), config script, integration change, data fix, and manual as the small residue that gets a click-by-click checklist.
- **Gate 2, production.** One human approval per deploy. A production write can never happen silently.
- **New policy.** No build starts until client approval is verified on the referenced Asana task. Scripted work gets a dev-team review before it comes back to you for testing.

**WHAT YOU SAY**

```
> Let's start execution. Work on tasks
O2C.3 and W.1.

> Do Phase 1.

> Pull the Asana task 'Price tag reprint
bug' and plan the fix.
```

# Retro at close: the learning loop

- Finals shipped, phase done: "Run the retro." Every learning gets walked with you and gets a decision: promote to doctrine, keep engagement-specific, or discard.
- Actual hours go back to LOE_BASELINES.md. Patterns go to NETSUITE_PLAYBOOK.md. Tool findings go to CAPABILITIES.md. Products and fit signals go to PRODUCTS.md.
- A business decision never becomes playbook best practice unless you promote it at the triage.
- Same habit at end of day: "Wrap up: update CONTEXT.md with today's decisions and what's open for next session."
- If deliverables/ holds finals and working/retro.md does not exist, the engagement owes its retro and the next session will offer it first.

# Six human gates, one screen

| # | Gate | What is being approved | Why here |
|---|---|---|---|
| **1** | Agenda | The onsite one-pager, before it reaches the client. | First thing the client sees |
| **2** | Findings | The complete candidate register, evidence log and verification log, before a word is drafted. | Materiality is a human judgment |
| **3** | Draft | The humanized drafts, reconciled against the candidate register so nothing captured has quietly vanished. | Last look before it becomes a file |
| **4** | LOE and commercial | Every hours figure, every priority, and the billing language, line by line. | Numbers become binding once sent |
| **5** | Execution: plan | The implementation plan for a task, before any build starts. | Design risk |
| **6** | Execution: production | Each production deploy, individually. | Live-account risk |

# Enforced, not trusted

| Mechanism | What it does |
|---|---|
| **Write guardrail** | Intercepts every NetSuite create, update and delete. A sandbox connection passes. A production connection forces an explicit human confirmation on every single write. A connection whose environment cannot be read from its name also stops. No production write happens silently. |
| **Render gate** | Blocks a deliverable from rendering while the client-facing drafts still carry mechanical copy tells. Installed per client folder. Fails open, because a broken gate must never stop a delivery. |
| **Copy scanner** | A deterministic scan behind the humanize pass. Every hit is cleared or consciously waived with a reason, and waived hits get raised at the review. |
| **Engagement log** | Generated from what each engagement actually wrote to disk. Status reflects work done, and folders that drift from the standard shape get flagged. |

# 03

# The weekly executive summary

The status document that catches what slipped. Run before every status call, from the client's folder.

Power Cloud
CONSULTING

# What it is

- Run from the client's folder, on the client's cadence, before every status call.
- Five sources: Asana, NetSuite hours, Gmail, the client's Drive folder, and meeting notes. Ask for a summary and it goes and gets all five.
- The recap is not the point. **The cross-reference is**: every meeting action item checked against the Asana open-task list, and the commitments that never became tasks listed with owner and meeting date. That is consistently where work slips.
- Output: a client-ready summary you review, plus the two follow-ups that turn it into action: a Gmail draft to the client contacts, and the missing Asana tasks created.
- No client folder, no summary. It says so and offers to set the client up rather than inventing a record.

**WHAT YOU SAY**

```
> Run the weekly executive summary for
Teppermans.

> What happened on 44 Pro in the last
two weeks?
```

# Two windows, both stated on the page

## Narrative window: through today

- Asana, Gmail, Drive and meeting notes are live to the minute.
- What happened, up to the moment of writing.
- Decisions, blockers, per-person action items, correspondence, documents delivered.

## Hours window: the trailing two full weeks

- Monday through the most recent completed Sunday.
- Time entries are not reliably submitted until the Monday after the week they cover.
- The Monday job writes each client's weekly hours into their own folder, so the summary reads hours locally. No NetSuite seat needed.
- Never extend the window to today. A partial week reads as a real number while being understated, which is worse than a number that stops where the data does.

**Hours lag up to a week. Never read a Friday summary's hours as current.**

# Two documents, not one

## working/exec_summary_{date}.md: internal, never sent

- The full picture: the blunt health assessment, anything mined from internal-only meetings, every gap the cross-reference found.
- Every material claim tiered: Verified, Reported, Inferred. The health verdict is Inferred and says so.
- States which hours path was used and the data's as-of date, every time.
- Drafted first and reviewed with you before anything renders.

## deliverables/{Client}_Executive_Summary_{date}.docx: client-facing

- Drafted from the internal one after you decide what carries: the health assessment as written, softened, or out.
- Rendered branded through the OS toolchain, humanized, QA'd before delivery.
- Safe by construction: separating the two means nothing candid reaches a client by accident.

**No exec summary reaches a client without a consultant reading and approving it first. Permanent policy, not a training-wheels phase.**

# It also shows you your own data quality

- "No next steps recorded" is the Asana hygiene talking, not the client's project. Absence in Asana is verified absence in Asana, nothing more, and the summary says so instead of reporting an artifact.
- A blank status update reads as stalled and produces a false at-risk health call. Post the update.
- A completed week inside the hours window with no entries from someone who normally logs to that client is a finding, flagged by name, before anyone invoices. An absent current week is not; that is the submission rhythm.
- The cross-reference is unflattering on purpose. A commitment made in a meeting and never logged as a task is exactly what it exists to catch.
- ASANA_CONVENTIONS.md is the minimum hygiene the summary depends on: a task per commitment, an owner, a due date, a status update posted.

**The summary is only as good as what you logged that week.**

# How a run goes

1

### Say the line

From inside the client's folder, in a named session.

2

### It gathers

Five sources. States the hours path and the as-of date.

3

### Internal draft

working/exec_summary_{date}.md, tiered, with the cross-reference.

4

### You review

Health verdict as written, softened, or out. What carries to the client.

5

### Client version renders

Branded .docx to deliverables/, humanized, QA'd.

6

### Send, then act

Gmail draft to the client. Missing Asana tasks created.

Live option: open a recent internal draft in working/ next to its client-facing .docx in deliverables/ and show what did and did not carry across.

04

# Three case studies

Imran, Justin, Ron. Ten minutes each. Real professional-services work: one client, one problem, one outcome.

Power Cloud
CONSULTING

# The four questions every story answers

**1**

What was the customer problem?

**2**

How did Claude help you solve it?

**3**

How did it help you implement the solution?

**4**

What about that was different or better than how you would have done it before?

Number 4 is the one the room remembers. Be specific: hours, days, the thing that would not have been caught, the client's reaction. In: true PS work. Out: development war stories, ideas, aggregates.

# Case study: Imran

| The customer problem | How Claude helped solve it | How it helped implement | Better than before |
| --- | --- | --- | --- |
| | | | |

Imran presents over the real client folder or deliverable. Ten minutes.

# Case study: Justin

| The customer problem | How Claude helped solve it | How it helped implement | Better than before |
| --- | --- | --- | --- |
| | | | |

Justin presents over the real client folder or deliverable. Ten minutes.

# Case study: Ron

| The customer problem | How Claude helped solve it | How it helped implement | Better than before |
| --- | --- | --- | --- |
| | | | |

Ron presents over the real client folder or deliverable. Ten minutes.

05

# Seven announcements

One or two sentences each. The how-to guide carries the detail.

Power Cloud
CONSULTING

# What changed

- **1. The OS moved.** It now lives at G:\Shared drives\Power Cloud OS, its own shared drive, everything at the root. The old Power Cloud Project Folders path is dead. Update pinned shortcuts today; the old drive stays as the read-only legacy annex for historical client folders.
- **2. The deck is the Presentation, not the Proposal.** We already have the work; the deck walks the client through findings. Filenames and documents now say Presentation.
- **3. Exec summaries are always human-reviewed.** No exec summary reaches a client without a consultant reading and approving it first. Permanent policy, not a training-wheels phase.
- **4. Client approval before build, dev review before testing.** No build starts until client approval is verified on the referenced Asana task; an email approval gets pasted onto the task with the source cited. Scripted work gets a dev-team review before it comes back to you for testing. Policy in effect now; the tool will enforce the gate soon.

# What changed, continued

- **5. Hours in exec summaries lag up to a week.** Each client folder gets a weekly hours log from the Monday job, so summaries read hours locally. The narrative is live to the minute; the hours window stops at the last completed Sunday. Never read a Friday summary's hours as current.
- **6. Feedback: when the tool annoys you, say so.** Type /feedback or say "log this as feedback" in any session; Claude writes the entry from what just happened and asks only what it cannot work out: time cost, blocked or not, every time or once. Everything gets triaged; nothing is silently deleted.
- **7. Session norms, the three that matter most.** One task, one session, started inside the client's folder. Name sessions client plus environment; paste the /rename line. You do not need the frontier model to write a PDF: escalate only when the default fails, and check your effort setting when a session feels slow. Hitting the $25 monthly credit limit is a signal, not a failure. Tell Mike.

# The ask: by end of next week, every consultant

- **Step zero verified on your machine.** Drive for desktop opening the new path, Git, Python with the four packages, brand fonts installed for all users, Claude Code signed into your Power Cloud account with the workspace opened once. Already a prerequisite for today; if it is not done, grab time on my calendar and we do it together.
- **One real task run through the OS**, in a properly named session, from a client folder. An exec summary before your next status call is the natural first one.
- **At least one /feedback entry** if anything got in the way. The tool only improves from what gets reported.

"Proofread everything before it leaves the building. 'The AI wrote it' is the failure, not the defense."

# Questions

- The workspace: G:\Shared drives\Power Cloud OS. Every session starts inside a client's folder there.
- The how-to guide: "How to Work in the Power Cloud OS" (PDF at the root). Part 5 is the menu of what to say.
- The longer capabilities guide: "The Power Cloud OS", linked from the how-to email.
- What is in flight: ENGAGEMENT_LOG.md. What is wrong with the tool: FEEDBACK.md. Both generated, read them, never edit them.
- Blocked on setup or access: Mike's calendar is open.

**The folder is the memory. You own what reaches the client.**

Power Cloud
CONSULTING

# A

# Appendix

Reference slides for the open floor and for reading later.

Power Cloud
CONSULTING

# The menu, functions 1 to 6: what to say

| Function | When | What you say |
|---|---|---|
| **1. Set up a client** | No folder here yet; new client or first new work for an old one. | "New client. Run session start." |
| **2. Prep a kickoff or onsite** | After the sales handoff, before you are in front of the client. | "Prep me for the kickoff call, and draft the questions we should send in advance." |
| **3. Run the assessment** | Discovery sessions done, transcripts in source/. | "The interviews are complete. Produce the handover docs." |
| **4. Execute approved work** | The client signed off; time to build. | "Let's start execution. Work on tasks O2C.3 and W.1." |
| **5. Work a support ticket** | Ongoing clients, one-off fixes, the daily queue. | "Pull the Asana task 'Price tag reprint bug' and plan the fix." |
| **6. Generate the exec summary** | Before every status call, on the client's cadence. | "Run the weekly executive summary for Teppermans." |

# The menu, functions 7 to 11

| Function | When | What you say |
|---|---|---|
| **7. Draft client communications** | Status updates, proceed-gates, billable-rework notices: wording with commercial weight. You send; it drafts. | "Draft a billable-rework notice for the reprint change. Factual, not apologetic." |
| **8. Do data work** | Shopify or QuickBooks in the stack, or a migration on the table. Profile, design decisions, mapping spec, dry run, gated load. | "They sent their item master export. Profile it and flag the design decisions before we map anything." |
| **9. Ask anything with full context** | Constantly. This is what folder-as-memory exists for. | "Based on everything in this folder, what should I know before tomorrow's call with their CFO?" |
| **10. Close out and learn** | Finals shipped, phase done, or end of day. | "Finals shipped. Run the retro." |
| **11. Tell us the tool got in your way** | The moment it happens. The annoyance is forgotten by end of day. | "Log this as feedback." or `/feedback` |

# Session habits that separate the people who get a lot out of it

- **One task, one session.** Files always survive; clearing erases conversation memory, never work.
- **Name your sessions** and **run several at once**: different clients, one task each. Named sessions are what make that manageable.
- **Brainstorm and plan together first.** Talk through the problem, challenge its approach, let it interview you. Your judgment matters most in the plan.
- **Then give it a chunk and walk away.** "I'm away for two hours; work the plan, and where you'd normally ask me, use your best judgment and log the decision." Never watch it type; waiting is not billable.
- **Correct it out loud** and tell it to remember. That is how a correction becomes a rule instead of next week's repeat.
- **Read everything before you trust it.** Polished output at speed is exactly what makes skipping review tempting. When a guardrail prompt appears, read it.

# Asking in a way that gets honest answers

- **Context without leading.** Give everything you know, but not your preferred answer. Once your idea is in the prompt, the objectivity is gone.
- **Current state before design.** Make it explain how the thing works today before it proposes anything.
- **"What's your source?"** for any claim that smells off. Never "are you sure?", which just invites an apologetic reversal.
- **Do not hand it the escape hatch.** Say "we can always check with the customer" and it will take that exit instead of reasoning. Make it commit; you decide afterward whether the client weighs in.
- **Steelman the alternative.** After it proposes one approach, ask for the strongest case for a completely different one.

# For the developers in the room

- Same folder, same guardrails. Build work lives in {Client}/execution/, sandbox first, SDF objects by default because they are versioned, diffable and revertible.
- **sdf-scaffold** stands up a SuiteCloud project to house standards; **sdf-deploy** validates and deploys with the account identity check and the two gates. SDF deploys on its own token, separate from the MCP connection.
- **ScriptReader**: four read-only tools (readScriptSource, listScripts, getScriptDeployments, readFile) plus the standardized MCP connection role. Standard install on every engagement, with the client's go-ahead, never deployed without it.
- Any net-new script, application or integration owes a **Use Case** before build and an **FDD & TDD** handover at ship. The customization-docs skill writes both, from any engagement type.
- **Dev review.** Scripted work gets a build-quality review before it returns to the consultant for testing. Policy now, tooling to follow.
- **Celigo** has its own skill: read-only flow inventory in discovery, error triage, and flow changes under the production gate. Shopify's own seventeen skills are installed rather than rewritten.

# Gotchas and quick fixes

- **Paths**: quote them; the workspace path has spaces.
- **File looks empty**: Drive has not synced. Right-click, "Available offline".
- **python or git not recognized**: PATH issue; new PowerShell window or reinstall with the PATH box checked.
- **Wrong fonts in a render**: fonts installed per-user only. Install for all users; the variable Montserrat font substitutes unreliably, use the static weights.
- **Wrong NetSuite account**: Claude stops and asks. Do not override; fix the connection.
- **Sandbox looks quiet**: sandbox data is frozen at the last refresh. Not evidence of a production problem.
- **Never put client data in a web search.** Public identity (their website, history) is fine and expected; account data, configs and financials are not.
- **Usage limit reached**: tell Mike. Not a failure; it is how seat sizes get right-sized.

# The rules that never bend

- **1. Read-only during discovery.** No writes to any client system during assessment or discovery. Writes happen in execution, sandbox first, human-gated.
- **2. No production write ever happens silently.** Every prod write forces an explicit confirmation. During discovery, the answer is decline.
- **3. Credentials never touch this Drive.** Tokens and secrets live on the connection holder's machine.
- **4. Everything client-facing gets human eyes before it ships.** You own what you send.
- **5. LOE numbers are commercially binding once sent.** Claude drafts them, a human owns them, line by line.
- **6. Consent is a record.** AI/MCP disclosure, installs and write authorizations get logged in CONTEXT.md with dates.
- **7. Check before create.** Folders, Asana projects, email groups: look for the existing one first.